

Deep Dive: Advanced Ontology | DevCon 5

Transcribed by [TurboScribe](#). [Go Unlimited](#) to remove this message.

Hi, everyone. Thanks for coming. I'm Landon.

I'm a group lead, and I work on the ontology here at Palantir. Hopefully you've heard of it. Today we're going to be talking about some advanced ontology features, but really we're going to be talking about mastering the underlying primitives.

We've been adding a lot of new features to the ontology lately, some you may be aware of and have seen at DevCon launches, some you might not have seen yet. You also might not understand the intention behind these features or quite how they all fit together. This session is going to walk through the features, why we've built them, and how we are putting them together.

This is going to be a deep look at some of our problem-solving, showing how we are turning problems into products. Sweet. Let's talk a little bit about the journey of the ontology.

We started with golden tables. This is about getting your data integrations right. Pulling all of your data from the disparate sources, collecting it in one place, making it easy to use, and eventually getting it into the ontology.

Next, we step to operational decision-making. This is about capturing the kinetics of your organisation through things like actions, logic, functions, et cetera, and integrating all of the models as well. This is about taking data into the system, doing compute inside the system, making those operational decisions, and then getting it out of the system into the real world where it can have real effects.

And finally, we moved to AI first. Now we're starting to integrate LLMs into the system, we're starting to layer on automation that is going to be mechanising all of those kinetics that you've already captured in your actions, functions, et cetera. Before we dive into the features, I also want to take a look back in history towards Gotham, which was Palantir's original product.

Gotham is used to power all of our intel and defence applications, still is, and we have a long history of developing the Gotham product. In the past, a lot of the intelligence workflows made very, very heavy use of unstructured data. You can think about reports coming in, PDFs, documents, et cetera, and analysts sitting toiling for hours trying to extract that data out of them.

And Gotham was built to help them ontologise that data, help them capture that value. So my point here is that we've been at this for a long time, and we're very good at helping you capture that nuggets of information out of your unstructured data, and with the advent of AI first workflows, LLMs themselves are very good at capturing this unstructured data. Overall, the ontology that we've built out through Gotham and now through Foundry and then AIP has been battle-tested and has a huge amount of depth of features, as you've seen throughout this

presentation and in prior DevCons.

We don't really think that any AI lab is going to magic an ontology into existence that's as deep, well-built, or battle-tested as this one. Okay. As we go into the features, I'm going to be hitting on these four ontology design principles, and this is really the crux of what I want you to take away from this presentation.

These ontology design principles are lessons that we've learned from the field over many years of tradecraft. These are the things that our four deployed engineers are using and thinking about when they're helping you to design your ontologies. We've broken this down into four principles for this presentation.

First, and most importantly, we have domain-driven design. This is about capturing the semantic intent of your data. This is about the fact that most of your real-world objects should be represented one-to-one in the virtual twin that is your ontology.

This is not a virtual twin of your data sets into the ontology, this is a virtual twin of what really exists out in the world. By structuring your data and your ontology to capture all of this semantic information with objects that make sense, links that make sense, you're going to find it much more intuitive to use, and also your agents are going to find it much more intuitive to use. Next, we have don't repeat yourself, using the rule of three, which is to say if you've built the same thing three times, you should probably try to refactor and capture that common workflow or that common data.

Again, this is going to help both you to navigate your ontology and your agents to navigate your ontology. So this is going to help a lot with context management rather than having to sift through 12 object types that all look more or less identical, but maybe are used for slightly different niches of data or by slightly different teams at your organization. If you capture all of that into one object type or into interfaces, as we'll see later, you can avoid repetition and it will be much easier for you to manage and figure out what is the canonical representation of this data, what is the canonical workflow, and again, this will help your agents with context management.

These next two are going to borrow some language from Java. I'm a Java developer, so bear with me for a second here. We have open for extension and closed for modification, typically shortened to open-closed.

What this means is that you are capturing the nuggets of your workflow in a way that you've kind of like locked away this most critical core pieces, right? That's closed for modification. No one's going to like accidentally break your core workflows. At the same time, you want that to be open for extension so that other developers in your enterprise can build off of your core data model.

So they're going to extend it with new workflows, with new types of data that fit into common interfaces, as we'll cover later. Next we have producer extends consumer super, PEX, also

known as covariance and contravariance. Again, it's a bit of a Java term, so bear with me, but basically what that means is that you can have a plug-and-play ontology, which is to say if you've designed things on interfaces, you can extend them and you can consume from things that, you know, you can use super classes to consume them as well.

You'll also see a couple of books down here that I would recommend on the topic overall. And for all of the software developers, what you're probably realizing is that these four ontology design principles that I've just laid out are, in fact, software design principles. This is really one of the key insights you should take away.

The ontology is effectively the software that's powering your organization. Because of this, we can borrow lessons from the long history of programming language research, software engineering research. I've read actually some really exciting papers about it, and those are the types of open source projects and places that we see this, you know, not just in Palantir's ontology, but elsewhere in the world, and we can draw lessons from all of this open source intelligence, everything else that people have been trying.

In summary, over the time and battle testing of all of our various deployments, all of our various enterprises across government, across commercial, across lots of different regions of the world, these are the four principles that I wanted to focus on today. They've been, as I said, proven themselves over time, they've been battle tested, and they're the principles that we've found the most success with when designing the most functional ontologies in the world. I wanted to start with interfaces, which we've talked about at a couple of previous DevCons.

The idea of interfaces is that you can have interface-powered workflows that then work off of any of the object types that implement those interfaces, either all together or one at a time. So in this example here, we've got some interface-powered workflow that's going to be built on the building interface, and you can see that we've got restaurants, office buildings, and arenas as three different object types that implement the building interface. And so either we can apply our workflow across all three of them all at once, or we can kind of have a picker maybe when we go into this workflow about, oh, I'm going to be looking at restaurants today, and I don't need to build three copies of the same app for each of these three workflows, or each of these three data environments.

You can also see that over on the right there, arena also extends schedulable resource. This is important. The Foundry ontology is multi-inheritance, so you can use the arena in both workflows built around buildings or workflows built around schedulable resources.

Say that you're trying to schedule a game, you might need to schedule some events as well as a building. You can use the arena for both of those. Not really shown here, but I also wanted to highlight that interfaces are multi-level, so you can have interfaces that extend other interfaces for more complex workflows.

Okay. Let's tie that back to our ontology design principles. First and most importantly, we have domain-driven designs.

Interfaces are going to let you think abstractly without hyperfixating on any specific dataset quirks that you might have. One of the worst patterns that I see, not to call out anyone that's done this before, but is to go through Pipeline Builder and just kind of click next, next, next, ontologize, and click map all of my properties. That's going to result in you effectively just copying whatever your upstream dataset is, which is fine as a starting point, great as a starting point, but it doesn't allow you the time to actually step back from the problem and think through your workflows from first principles.

Overall the goal is that with a well-designed ontology, using it should feel very intuitive. You should be able to enter a flow state when you're trying to work with it, as well as your agents. You should have API names that make sense, you should have links that make sense.

The more that your ontology represents what you're actually dealing with in the real world, the more flexible and scalable you'll find it to be. Next interfaces help you to not repeat yourself, right? I've talked about how this buildings interface lets us build a buildings workshop, and then that can be used for our three different object types, right? So interfaces make it much easier to avoid workflow duplication. Next, open for extension and close for modification.

So you can imagine that this building interface is something that's like core to our enterprise. It's been battle tested, it's been refined over time. We figured out what are the critical properties here, what are the critical links that it might have.

We want to lock those down. So we're going to close that interface for modification so that no one accidentally messes it up, while still leaving it open for extension. So new other builders in your organization can extend it with new workflows, new object types, et cetera.

Because the ontology is multi-inheritance, you also can favor composition over inheritance. Again, this is a really common software debate, composition versus inheritance. And for our example of the sports arena, you can see that it extended both building and schedulable independently, effectively composing those two topics, rather than trying to inherit this weird, schedulable building object or interface that you might have otherwise.

Finally, we have producer extends consumer super, again, called covariance and contravariance. If you go look that up on Wikipedia, you'll find that this is a pretty common programming language term. Effectively what this means is that you have flexibility when you use interfaces.

So if you've built an OSDK app or a workshop, for example, that works off of object sets of things that extend event, so like question mark extends event, then you can safely pass in an object set of NBA games or an object set of DevCon keynotes. On the other side of that, for super, this is a bit of a Java terminology, but if you're trying to call a function on DevCon keynote, which is to say,

like, consume a DevCon keynote object, you can safely call reschedule DevCon keynote, or you can call a function whose signature is reschedule question mark super event, right? So you can call it because DevCon keynote has a super class of event. Again, Java terminology, that doesn't make sense, that's fine.

Basically what this means is that interfaces gives you flexibility with your workflows and flexibility with your functions. Next I want to talk about structs. This is a pretty old UI, and I basically just wanted to hammer home, again, we've been at this for a long time.

This is our original Gotham ontology manager, and this is the address struct property that we have there. And this is how the address struct shows up in the Gotham frontend. So you can see that the address is made up of multiple fields, even though it's a single property.

So we have Norgade 42, which is the street address, we have Copenhagen, which is the city, so these are two different fields on our struct. We're also capturing metadata. So you can see that under the sources, you can see that this is manually entered data as opposed to coming from one of our upstream data sources.

You can see who created it, when it was created, when it was last modified, et cetera. And all of this is metadata that we've captured directly alongside the property value as effectively additional struct fields. How does this show up in Foundry? You see something very similar now, right? You can define a struct property, in this case we have our same address on our office building example.

You can see that the address has a street address, city, state, and postal code. And then this also shows up in actions, where you get this fancy little dropdown that kind of indicates to you that these are grouped together. We've also found that structs are very useful in the advent of LLMs, right? As Colton was showing in his demo earlier, AIP logic does kind of behind the scenes capture some of the reasoning for you.

But sometimes it's really useful to ontologize that for further processing. So this is a real world example of a Slack helper bot, basically, that we've set up on one of our help ontology channels, where people ask questions. And you can see that out of this AIP logic, we're capturing both the response that we want to pass along to the user, we're capturing some metadata, like the most relevant Slack message if this question has been asked before, the most relevant doc URL, this is linked to our public docs page, as well as the reasoning that the LLM had behind its answer.

And we can use that for downstream processing. So if I actually go run this example, I ask how do I make a new object type, it gives me a response, and I'm capturing both, again, the response, most relevant Slack message, it didn't need one because this is a very straightforward question, we're capturing the most relevant doc URL, which is probably our page about object types, and we're also capturing the reasoning if we wanted to look at that later as well. Now you might be asking, this metadata, it kind of goes against some of our ontology design principles, where we

really want to focus on semantically relevant information.

That metadata is relevant, but it's not semantic, right? To bridge this gap, we're going to also introduce two new features, reducers and struct main fields. One of the most powerful features of Gotham is that every property can have multiple values. Could be coming from different data sources, could have different securities, could just be something that's been updated over time.

So for example, if you look at a person object and they have an address, everyone moves around, every person is going to have multiple addresses over time, so you're going to be capturing that history of the address property by just basically capturing it as a list. And the metadata about when they lived at each address is going to be metadata fields on the address struct. So what we're doing with reducers is we're pulling from that list of different addresses, in this case we've set it as like we're going to pull the most recent, the struct which has a most recent last modified on metadata property, and basically bubble that one to the top.

And what that means is that in many of our UIs, you'll see just that one address, right? That's the one that you want to focus on, and if you hover on it or in some UIs there's like an expand or like a show more type of button, you'll see the rest of the addresses. You can search across all of them, but when you don't need to think about the full history, you just get to look at the single one. A similar concept applies for struct main fields.

In this case we've got a geolocation, and the main field is just going to be that single geopoint property, but we also want to capture metadata like when it was created, who created it, and so on, and so what we've defined here is that we've defined the value, which is the geopoint property, as the only struct main field. So we can actually use this property, even though it's a struct, we can use it as if it was just a geopoint, right? We can use it in all of our geopoint workflows, it will show up as just a geopoint in the UI, and again, on hover or if you need to, you can look through the additional metadata fields to see that additional information. Bringing this back again to our ontology design principles, how do structs, reducers, and struct main fields fit into those? First we have domain-driven design, objects represent semantically meaningful concepts, and this is perfect to do with structs.

You don't have to separate each of the fields of the struct into different properties, right? This is a more semantic way to capture that data. And again, don't repeat yourself. Structs can also help capture the most important metadata in the most proximate location, which is associated with the rest of those properties.

Otherwise you might have to go pile those metadata fields onto a sidecar object or onto some other hidden properties. Okay. Next I want to talk about derived properties.

This is a very powerful feature. In this example here, we've got basically an org chart, so we've got my object up there, I'm Landon Carter, group lead, right? Team ontology. But we want to add

some additional fields, right? Peeps is basically our internal term for people that report to me.

So let's say in this example I have three peeps and their names are 1, 2, and 3, right? I could capture that data by adding additional properties to the worker object to just write them there. That would be called denormalizing the data. I'm copying it around so that it's very useful to use in place.

However, that can make the data very brittle, right? If one of my peeps decides to update their name, if I get a new peep, that data is already represented in the rest of the ontology through the links and the linked objects. With derived properties, I can leave my data in a normalized format, which is to say I leave all of the data where it normally would live, and I can define the semantic business logic around how I would infer those property values. So in this case, I've got my peep names, is I'm going to search around to my peep objects, and then I'm going to collect a list of the worker name property, so that's the names of all of my peeps.

For the peep count, I'm just going to count the number of linked worker objects where I have a lead peep relationship with them, right? So we're defining the business logic in our ontology to capture that in a very useful and productive way. Again, hammering home our ontology design principles, this is capturing domain-driven design, right? Derived properties make it much easier to avoid data duplication and represent your objects in a semantic way without having to deal with the brittleness of denormalizing your data. Okay.

This last section, I'm going to kind of fly through. This is a lot of very complicated ways to combine all of the features that we've talked about so far, as well as a few additional ones. And what I really want you to take home from this is the depth that you can reach with the ontology when you layer all of these features together.

You don't really need to understand these examples. I'm just trying to show you the depth that you can achieve. So one of the more powerful features of Gotham, as I said, is that properties can have multiple values at different security levels.

Let's talk about where you might want that in Foundry. We're going to have now, skipping to a different example, patients and data about them, right? So we've got a few patients here. And at our level zero security, we've just got data set back to security, right? All of these have the same security applied to them.

With row-wise security, which you can achieve through restricted views or with object security policies that Kevin and Laura were launching earlier, you can now say this row is secret, right? Alex Karp is a VIP. Not everyone at the hospital needs to know that he's there. But you see some additional information that might need to be restricted.

You can step this up into cell-level security by combining row-wise security and column-wise security, which, again, is what Kevin and Laura were talking about earlier. With OSPs, this is very

easy. You could also achieve this in the past with restricted views, plus multi-data set objects, MDOs.

And now you can see that the patient note section has different security for different objects, and you can see that that has a different security from other columns in our data set. You can step this up a level further, though, with derived properties to achieve sub-cell security. In this case, we've split the array of patient notes so that each element of the array has its own unique security level, and you can see that some of our more sensitive patient notes, like this patient suspects the spouse is poisoning him, can be locked away with a more secret marking applied to it.

And then you can go a level further. Gotham has a really powerful tool called entity resolution, where if you notice in your data that actually I have two objects, I realize this was a data integration error or I've just learned some new things, I want to resolve them to be a single object, you can do that as well. So in this case, we've seen that we have a Diana Mercer and we have a Diane Mercer.

We've realized this is actually just a single person. We should resolve them together. With derived properties, if you link it up in just the right ways, you can achieve this entity resolution concept so that when you open up either the Diana Mercer object or the Diane Mercer object, you see properties from both, and they carry with them their individual original securities.

This is a very complex data model. You don't need to understand it, but this is how that works under the hood, basically. We've got basically two layers of objects, which is in this example, like equipment and then equipment observations.

You can see a bunch of links between them about what's been resolved to what, and you can see a three-step chain on the derived properties for how to ensure that whether you're looking at equipment A or equipment B, you see the same data in either case, and if you were to resolve in a third, equipment C, you would see that that immediately updates with all of the data from all three equipments. Levels of security, again, ontology design principles empower you to do domain-driven design. You're expressing your security in a semantically precise way, right? Especially with object security policies, as you saw from Kevin and Laura's demo earlier, you could see that directly in the ontology, you have security policies with names that make sense and which protect specific properties or specific rows of your data.

Finally, I'd like to introduce object-backed link types. It's another one of our more complex features. In this, returning to our, like, org chart example, we have employees and we have ventures, which is the team that they're working on, and we also have a venture staffing which connects them, so this venture staffing object captures some metadata about when the employee worked on a particular venture.

Now, this is a kind of special case because sometimes in some workflows, you don't really care about the venture staffing, you just want to see what are all of the teams that this person has ever

worked on. For that, you want to take a link directly from the employee to the venture, and with object-backed link types, you can implement that link with the venture staffing object. On the other hand, in some workflows, you do want to step through the venture staffing object and filter based on those properties.

For example, if you want to find the current venture, that's the process that you would use to do that. How do object-backed links fit into our ontology design principles? Once again, domain-driven design, our most important principle, objects should represent semantically meaningful concepts, and the links between them should represent semantically meaningful concepts. Sometimes the venture staffing is semantically meaningful, sometimes it's just a helper object and it needs to be hidden.

With object-backed link types, you can achieve both. All right. I'd like to conclude by hammering again our four ontology design principles, which is domain driven design, don't repeat yourself, open closed, and producer extends, consumer super.

I think with that, I will close it out and take any questions. Thank you.

Transcribed by [TurboScribe](#). [Go Unlimited](#) to remove this message.